



## **Third Year Engineering**

### **18BTCS501-Design and Analysis of Algorithms**

#### **Class - T.Y. (SEM-I)**

#### **Unit - IV**

#### **Backtracking & Branch-n-Bound**

**AY 2023-2024    SEM-I**





## Unit IV - Syllabus

### Unit IV- Backtracking & Branch-n-Bound

**09 hours**

- Backtracking
- n-queen problem
- Sum of subsets problem
- Branch-n-Bound: Principle
- Strategies: FIFO, LIFO and LC approaches
- Travelling Salesperson Problem
- Knapsack problem





## Unit 4 - Outline

- **Backtracking**
- n-queen problem
- Sum of subsets problem
- Branch-n-Bound: Principle
- Strategies: FIFO, LIFO and LC approaches
- Travelling Salesperson Problem
- Knapsack problem

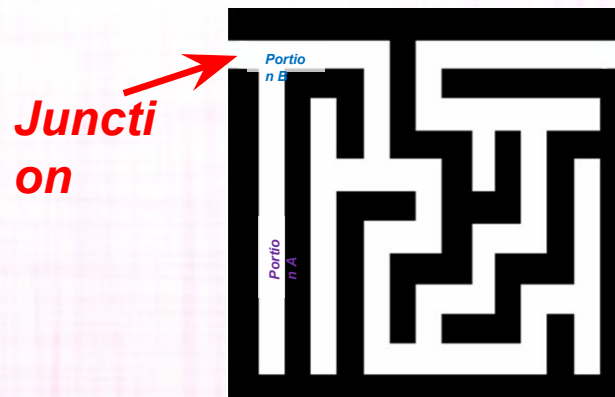






## BACKTRACKING: PRINCIPLE

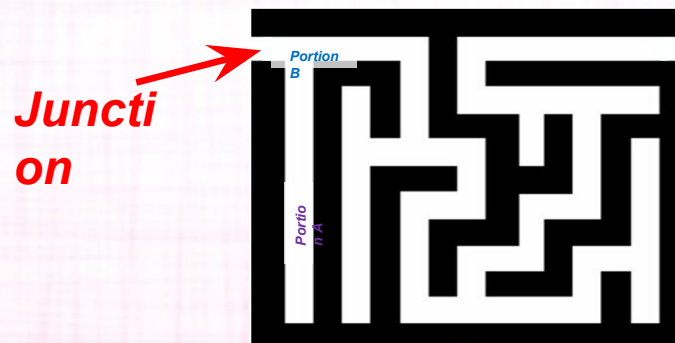
- Backtracking is a technique used to solve problems with a large search space, by systematically trying and eliminating possibilities.
- A standard example of backtracking would be going through a maze.
  - At some point, you might have two options of which direction to go:





## BACKTRACKING [CONT...]

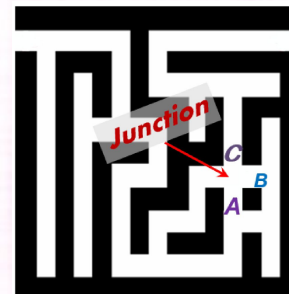
- One strategy would be to try going through Portion A of the maze.
  - If you get stuck before you find your way out, then you "backtrack" to the junction.
- At this point in time you know that Portion A will NOT lead you out of the maze,
  - So you then start searching in Portion B





## BACKTRACKING [CONT...]

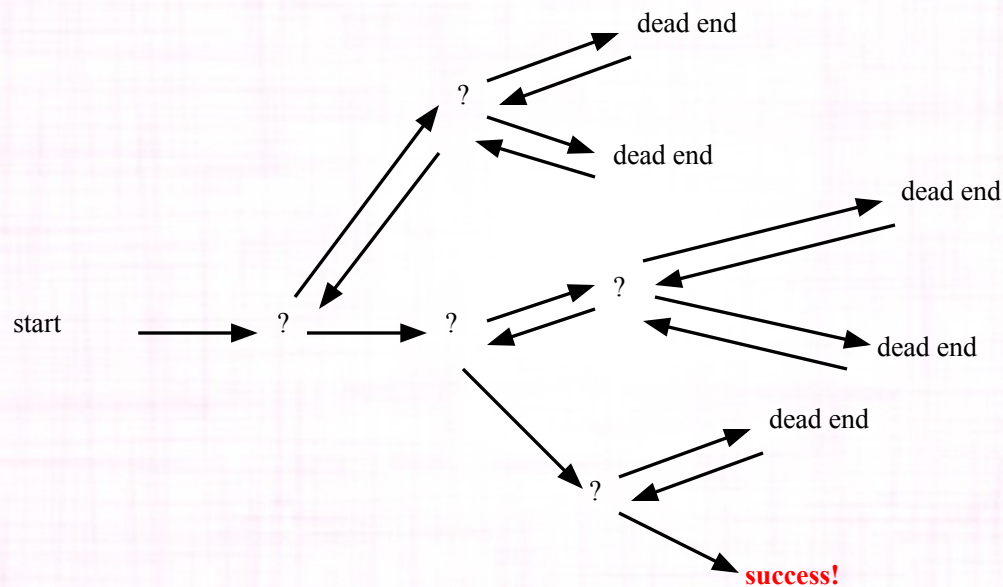
- Clearly, at a single junction you could have even more than 2 choices.
- The backtracking strategy says to try each choice, one after the other,
  - if you ever get stuck, "*backtrack*" to the junction and try the next choice.
- If you try all choices and never found a way out, then there is no solution to the maze.







## BACKTRACKING [CONT...]





## Unit 4 - Outline

- Backtracking
- **n-queen problem**
- Sum of subsets problem
- Branch-n-Bound: Principle
- Strategies: FIFO, LIFO and LC approaches
- Travelling Salesperson Problem
- Knapsack problem







## N-QUEEN PROBLEM

- Problem surfaced in 1848 by chess player Max Bezzel as 8 queens (regulation board size)
- Premise is to place N queens on N x N board so that they are non-attacking
- A queen is one of six different chess pieces
- It can move forward & back, side to side and to its diagonal and anti-diagonals
- The problem: given N, find all solutions of queen sets and return either the number of solutions and/or the patterned boards.





## BASIC ALGORITHM: N-QUEEN PROBLEM

- In this problem, a chessboard will be given of  $4 \times 4$  (i.e., 16 cells)
- For  $4 \times 4$  chessboard, we have 4 queens to place in the chessboard.
- So, just like in the game of chess, a queen can move in horizontal (i.e., row-wise), vertical (i.e., column-wise) and diagonal direction.
- So, according to our problem, we have to place all the 4 queens in such a way that no two queens are under attack.
- When two queens are in the same row/column/diagonal, we call they are under attack.



### BASIC ALGORITHM: N-QUEEN PROBLEM [CONT..]

- Generate a list of free cells on the next row.
- If no free cells, backtrack (step 2 of previous row)
- Place a queen on next free cell & proceed with step 1 for next row
- If no next row, proceed to step 3
- At this point, you have filled all rows, so count/store as a solution

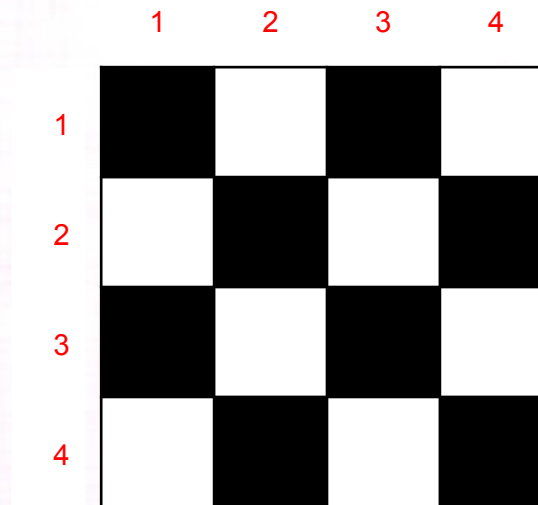






## BASIC ALGORITHM: *EXAMPLE*

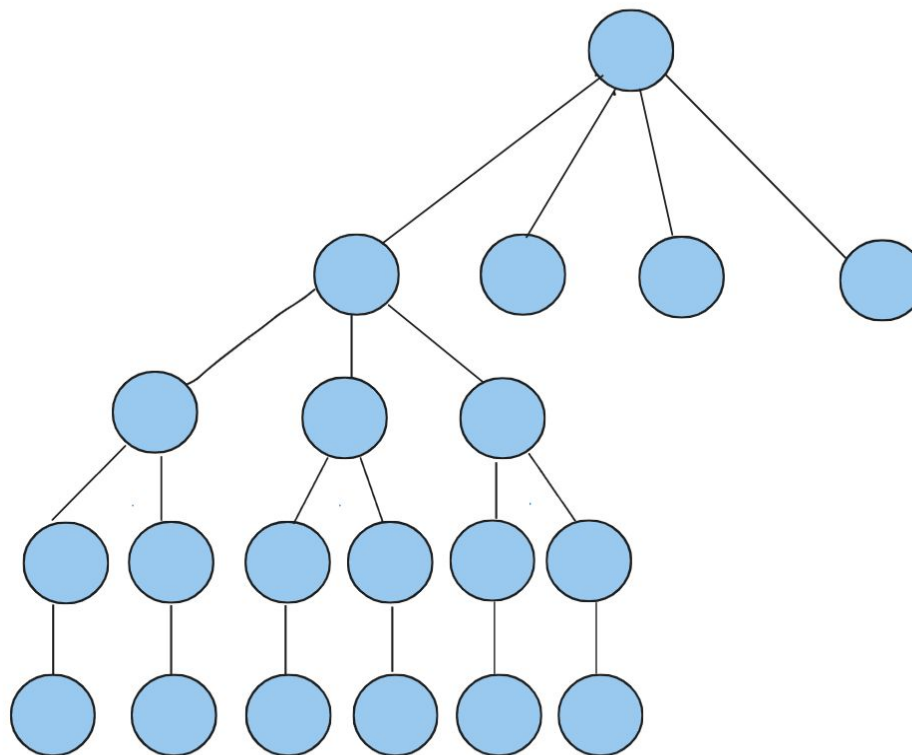
- Total no. of ways by which we can arrange all the four queens in the 16 cells are  ${}^{16}C_4$  ways





## BASIC ALGORITHM: *EXAMPLE*

So, let us generate a state-space tree:

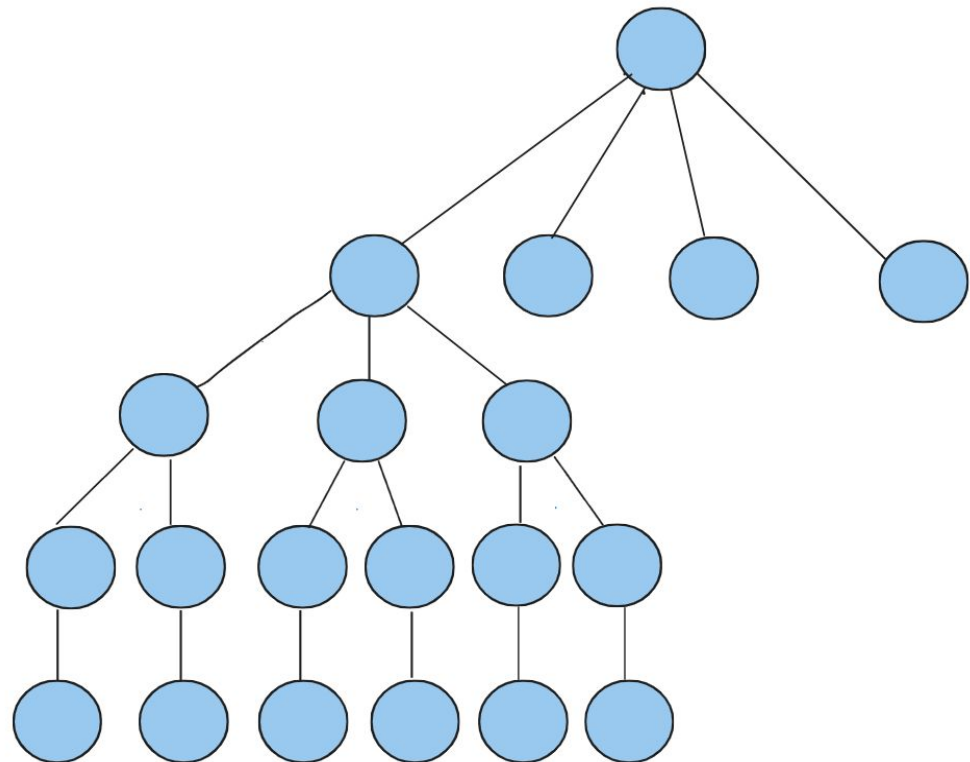




## BASIC ALGORITHM: *EXAMPLE*

So, let us generate a state-space tree:

Total number of nodes=

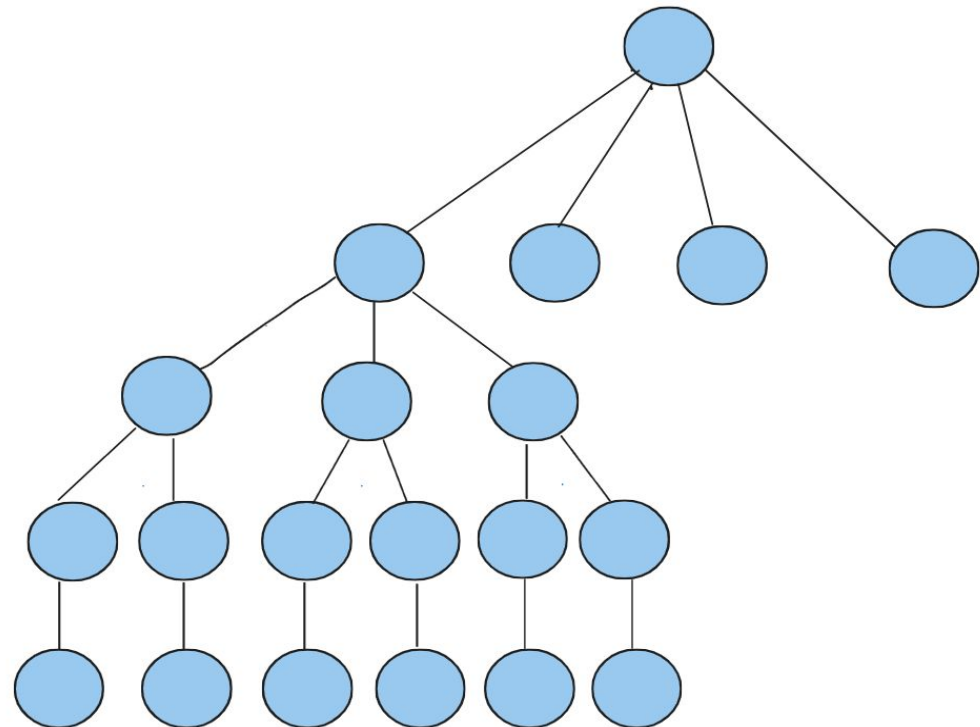






## BASIC ALGORITHM: *EXAMPLE*

So, let us generate a state-space tree:





## BASIC ALGORITHM: *EXAMPLE*

|   |   |   |   |   |
|---|---|---|---|---|
| F | F | F | F | F |
|   |   |   |   |   |
|   |   |   |   |   |
|   |   |   |   |   |
|   |   |   |   |   |



|   |  |  |  |  |
|---|--|--|--|--|
| Q |  |  |  |  |
|   |  |  |  |  |
|   |  |  |  |  |
|   |  |  |  |  |
|   |  |  |  |  |



|   |  |   |   |   |
|---|--|---|---|---|
| Q |  |   |   |   |
|   |  | F | F | F |
|   |  |   |   |   |
|   |  |   |   |   |
|   |  |   |   |   |



|   |  |   |  |  |
|---|--|---|--|--|
| Q |  |   |  |  |
|   |  | Q |  |  |
|   |  |   |  |  |
|   |  |   |  |  |
|   |  |   |  |  |





## BASIC ALGORITHM: EXAMPLE [CONT...]

|   |  |   |  |   |
|---|--|---|--|---|
| Q |  |   |  |   |
|   |  | Q |  |   |
|   |  |   |  | F |
|   |  |   |  |   |
|   |  |   |  |   |



|   |  |   |  |   |
|---|--|---|--|---|
| Q |  |   |  |   |
|   |  | Q |  |   |
|   |  |   |  | Q |
|   |  |   |  |   |
|   |  |   |  |   |



|   |   |   |  |   |
|---|---|---|--|---|
| Q |   |   |  |   |
|   |   | Q |  |   |
|   |   |   |  | Q |
|   | F |   |  |   |
|   |   |   |  |   |



|   |   |   |  |   |
|---|---|---|--|---|
| Q |   |   |  |   |
|   |   | Q |  |   |
|   |   |   |  | Q |
|   | Q |   |  |   |
|   |   |   |  |   |







## BASIC ALGORITHM: EXAMPLE [CONT...]

|   |   |   |   |   |
|---|---|---|---|---|
| Q |   |   |   |   |
|   |   | Q |   |   |
|   |   |   |   | Q |
|   | Q |   |   |   |
|   |   |   | F |   |



|   |   |   |   |   |
|---|---|---|---|---|
| Q |   |   |   |   |
|   |   | Q |   |   |
|   |   |   |   | Q |
|   | Q |   |   |   |
|   |   |   | Q |   |

DONE





## Unit 4 - Outline

- Backtracking
- n-queen problem
- **Sum of subsets problem**
- Branch-n-Bound: Principle
- Strategies: FIFO, LIFO and LC approaches
- Travelling Salesperson Problem
- Knapsack problem





## SUM OF SUBSETS

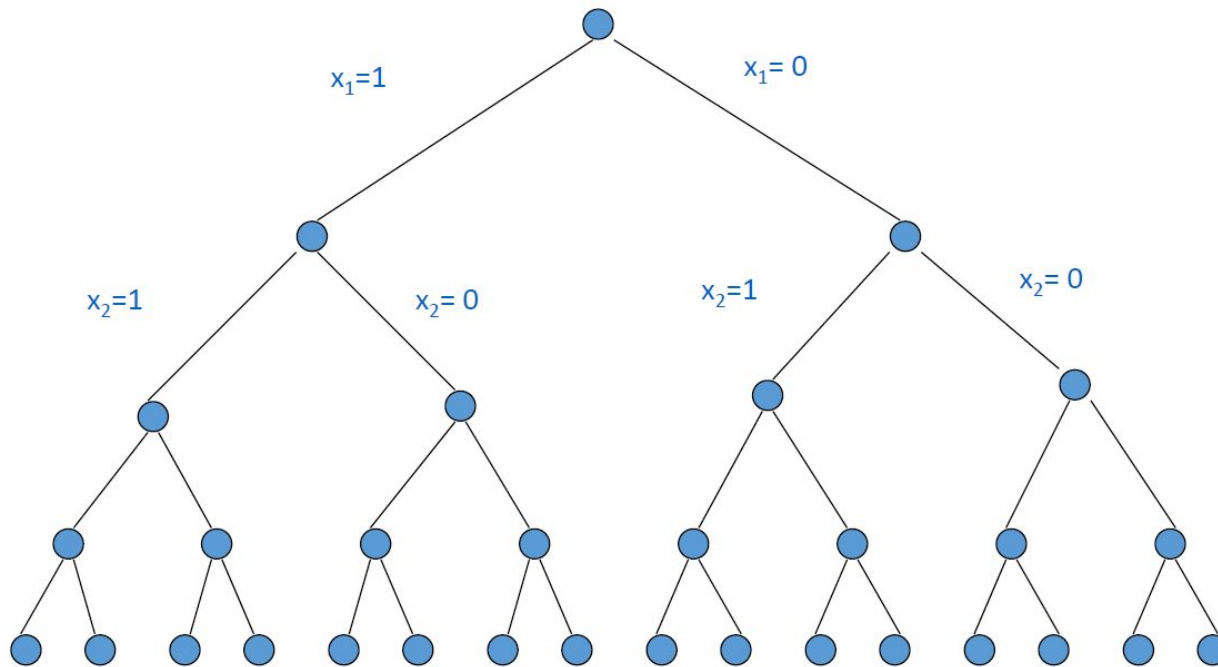
- In this problem, a set of weights will be given
- We have to take subset of those weights such that their sum is exactly equal to a value  $m$
- The solution set is represented as 0/1 form







## SUM OF SUBSETS [CONT..]



Each forward and backward moves take  $O(1)$  time



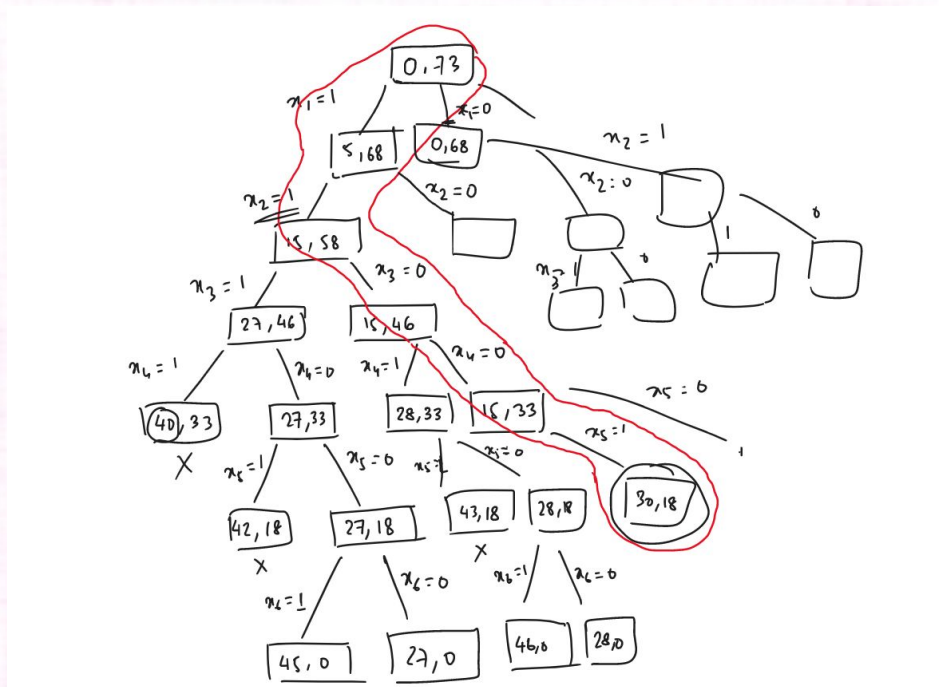
## SUM OF SUBSETS: *Example*

- $W[1:6] = \{5, 10, 12, 13, 15, 18\}$
- $n=6, m=30$ ; where  $n$  is the number of number of weights,  $m$  is the target value
- So, selection of the above weights can be done in various ways.
- Hence, let us try all possible ways of their selection and find out which of those selection is giving us the exact value of  $m$  (i.e., equals to 30)
- To solve the problem, we will have to draw a state-space tree for the problem





## SUM OF SUBSETS: Example [CONT..]







## Unit 4 - Outline

- Backtracking
- n-queen problem
- Sum of subsets problem
- **Branch-n-Bound: Principle**
- Strategies: FIFO, LIFO and LC approaches
- Travelling Salesperson Problem
- Knapsack problem





## BRANCH AND BOUND: PRINCIPLE

- **Branch and bound** builds the state-space tree and find the optimal solution quickly by pruning few of the tree branches which does not satisfy the bound.
- Backtracking can be useful when some other optimization techniques like greedy or dynamic programming fail. Such algorithms are typically slower than their counterparts.
- In the worst case, it may run in exponential time, but careful selection of bounds and branches makes an algorithm to run reasonably faster.



### BRANCH AND BOUND: PRINCIPLE [CONT...]

- Branch and bound approach is used to solve minimization problems and not maximization problems
- Anyway, we can convert a maximization problem to minimization problem and solve it.
- One example of minimization optimization problem is Job sequencing with deadline where we need to pay the penalty for not doing a job instead of earning profits for doing a job.





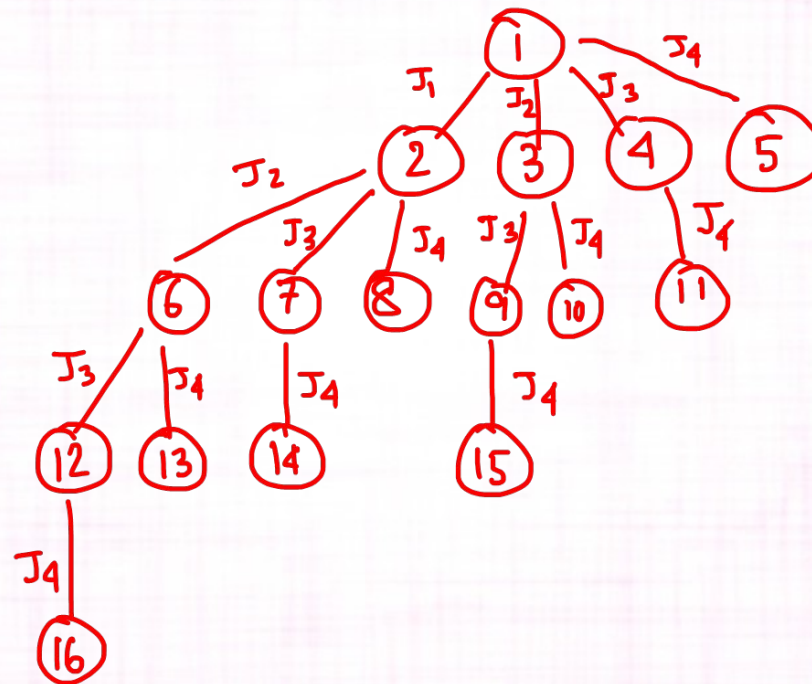
## Unit 4 - Outline

- Backtracking
- n-queen problem
- Sum of subsets problem
- Branch-n-Bound: Principle
- **Strategies: FIFO, LIFO and LC approaches**
- Travelling Salesperson Problem
- Knapsack problem





## BRANCH AND BOUND STRATEGIES: *FIFO*





Dept. of Computer Science &  
Engineering.



MIT-ADT  
UNIVERSITY  
PUNE, INDIA

## BRANCH AND BOUND STRATEGIES: *LIFO*







## BRANCH AND BOUND STRATEGIES: *LCBB*

- Instead of generating entire tree, we always pick-up the nodes with minimum cost/least cost and try to explore that one, so that quickly we can reach the solution.





## Unit 4 - Outline

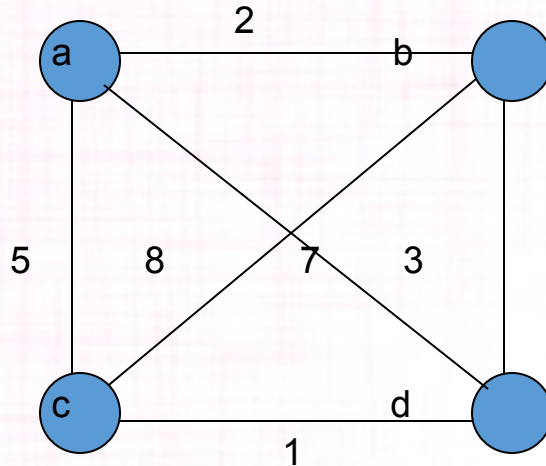
- Backtracking
- n-queen problem
- Sum of subsets problem
- Branch-n-Bound: Principle
- Strategies: FIFO, LIFO and LC approaches
- Travelling Salesperson Problem
- **Knapsack problem**



# Branch and Bound



## Traveling salesman problem



$\{ a, b, c, d \}$   
represents 4 cities

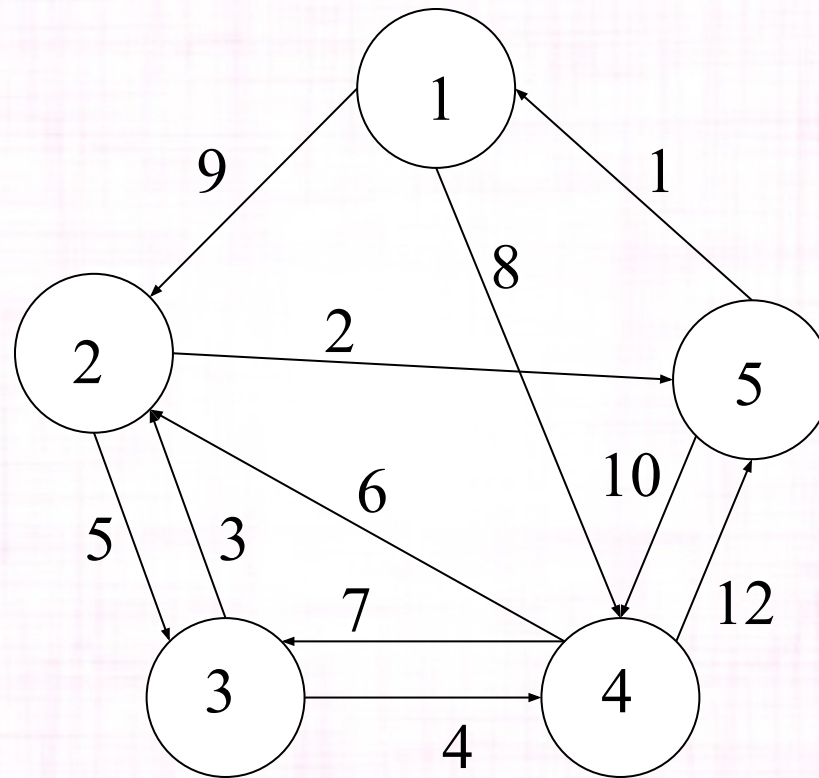
The weights  
represent distances  
between cities

**Problem:** find the shortest path from a city (say **a**), visit all other cities exactly once, and return to the city where it started (city **a**).





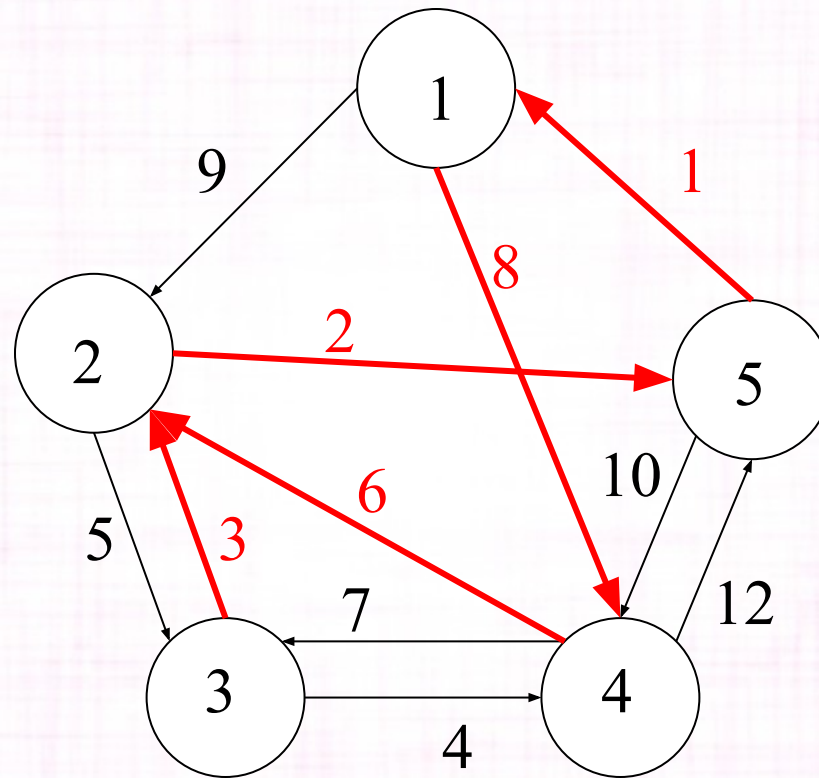
# Bound on TSP Tour



Every tour must leave every vertex and  
and arrive at every vertex.

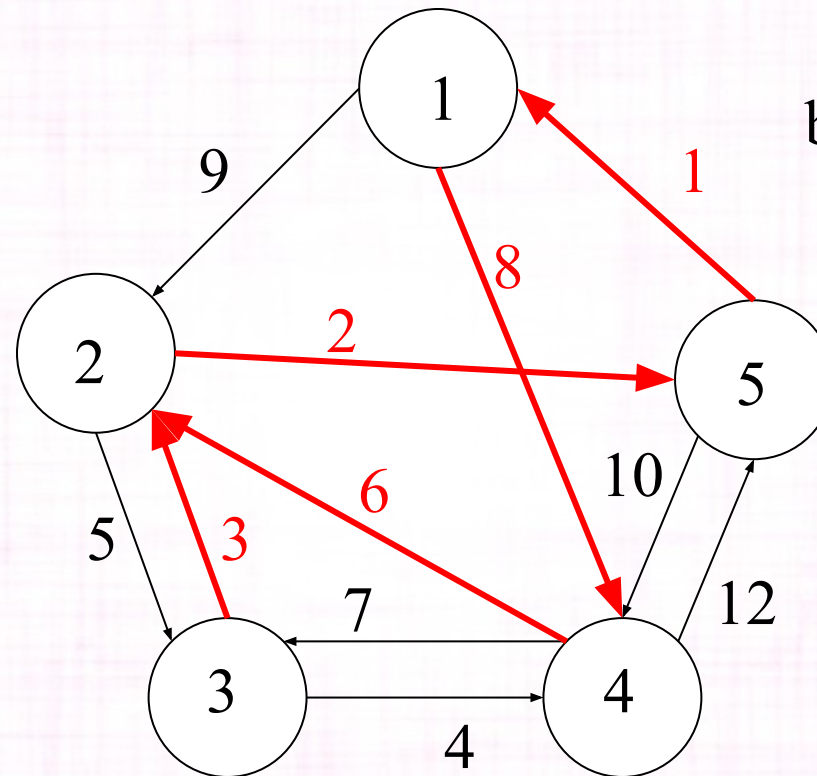


# Bound on TSP Tour



What's the cheapest way to leave each vertex?

# Bound on TSP Tour

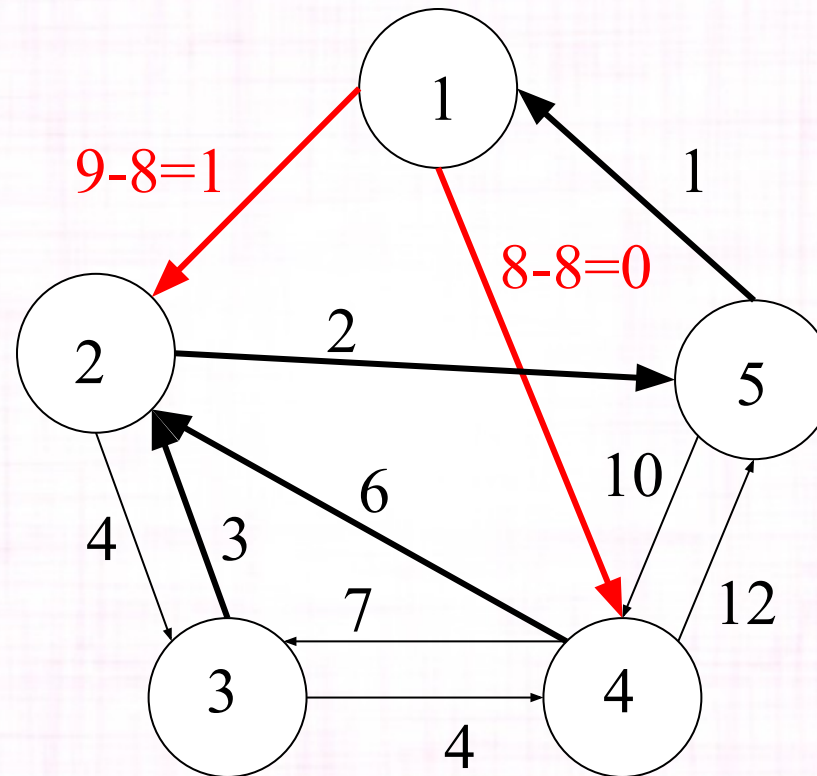


$$\text{bound} = 8 + 6 + 3 + 2 + 1$$

Save the sum of those costs in the bound (as a rough draft).  
Can we find a tighter lower bound?



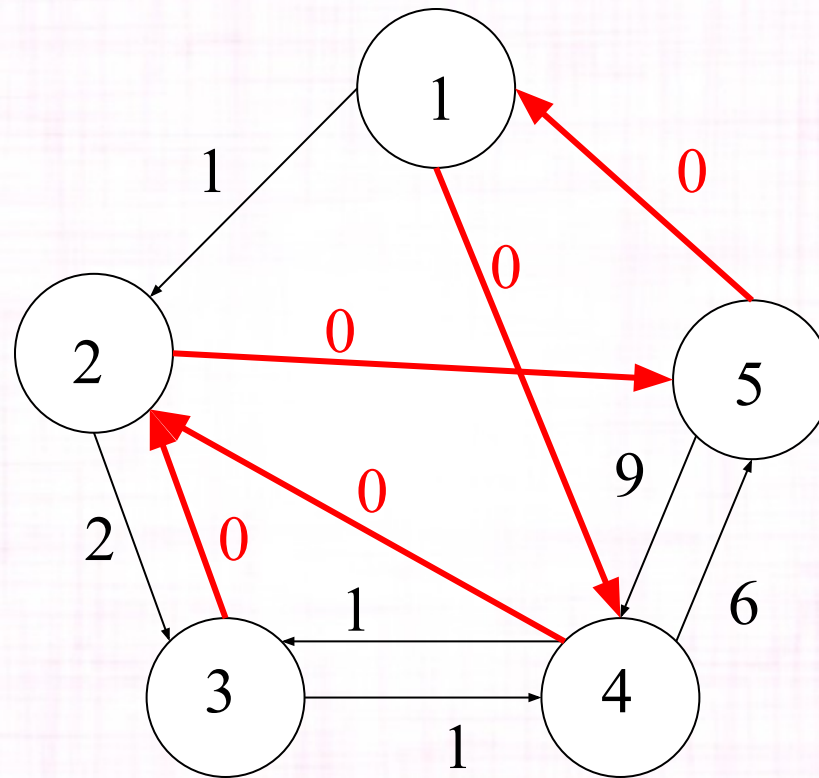
# Bound on TSP Tour



bound=20

For a given vertex, subtract the least cost departure from each edge leaving that vertex.

# Bound on TSP Tour

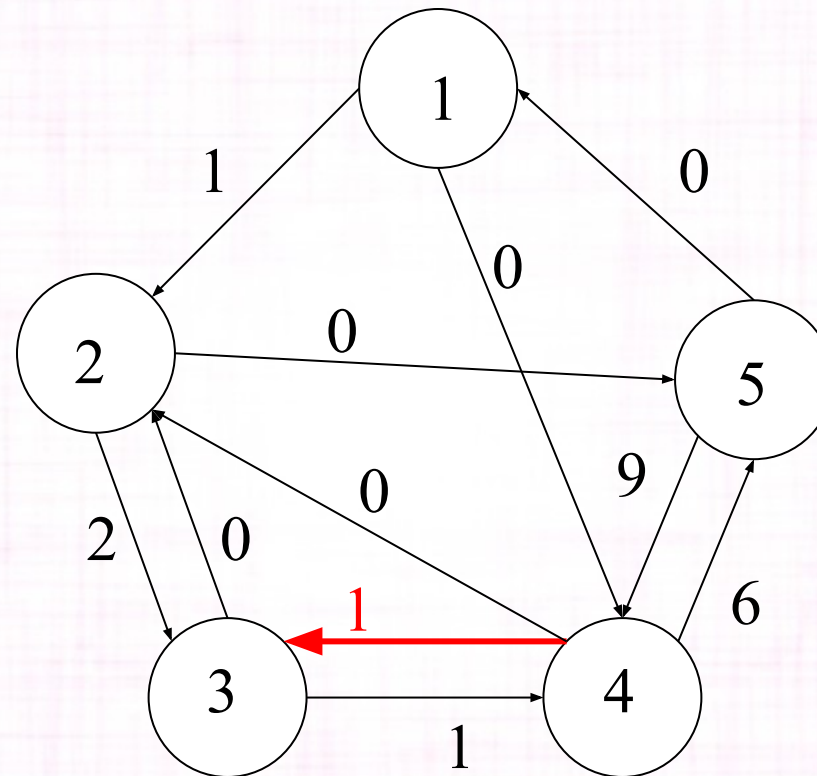


bound=20

Repeat for the other vertices.



# Bound on TSP Tour



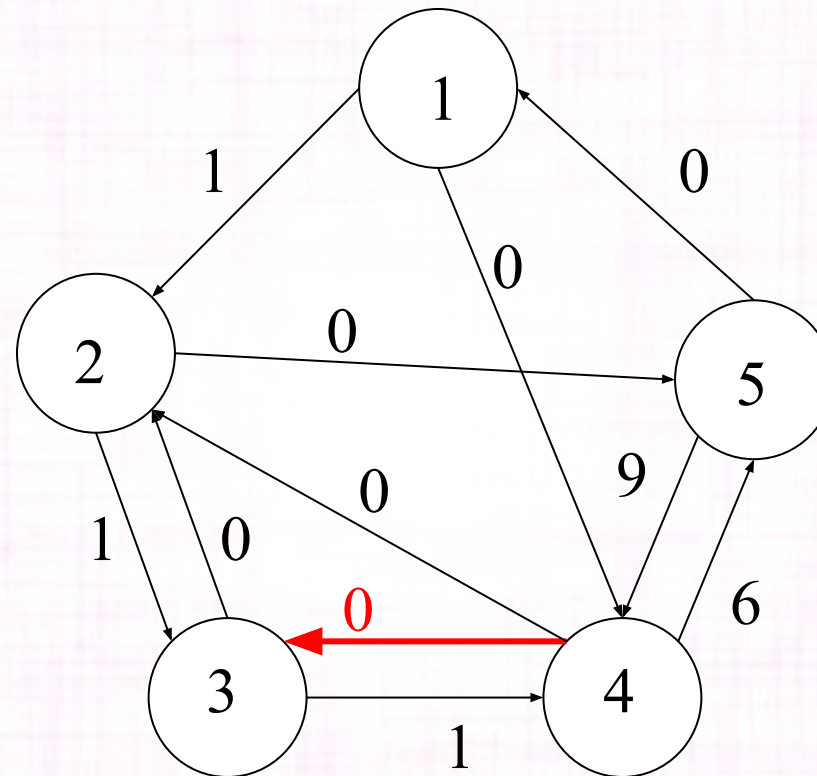
bound=20

Does that set of edges now having 0 residual cost arrive at every vertex?

In this case, the edges never arrive at vertex 3.



# Bound on TSP Tour



bound=21

We have to take an edge to vertex 3 from somewhere.  
Assume we take the cheapest. Subtract its cost from other edges  
entering vertex 3 and add the cost to the bound.

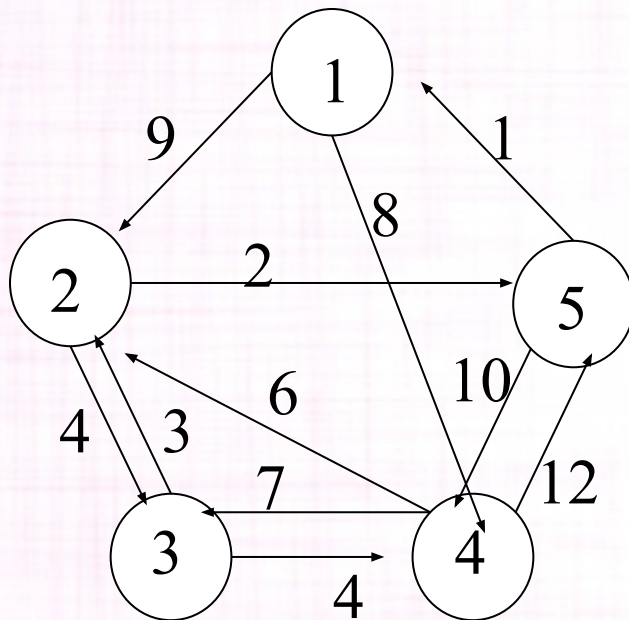
**We have just tightened the bound.**

# The Bound

- It will cost at least this much to visit all the vertices in the graph.
  - there's no cheaper way to get in and out of each vertex.
  - the edges are now labeled with the *extra* cost of choosing another edge.



# Bound on TSP Tour

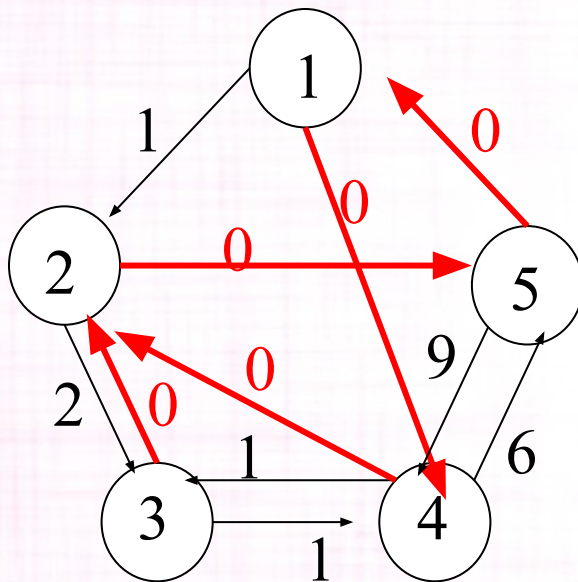


$$\begin{pmatrix} 999 & 9 & 999 & 8 & 999 \\ 999 & 999 & 4 & 999 & 2 \\ 999 & 3 & 999 & 4 & 999 \\ 999 & 6 & 7 & 999 & 12 \\ 1 & 999 & 999 & 10 & 999 \end{pmatrix}$$

Algorithms do this using a cost matrix.



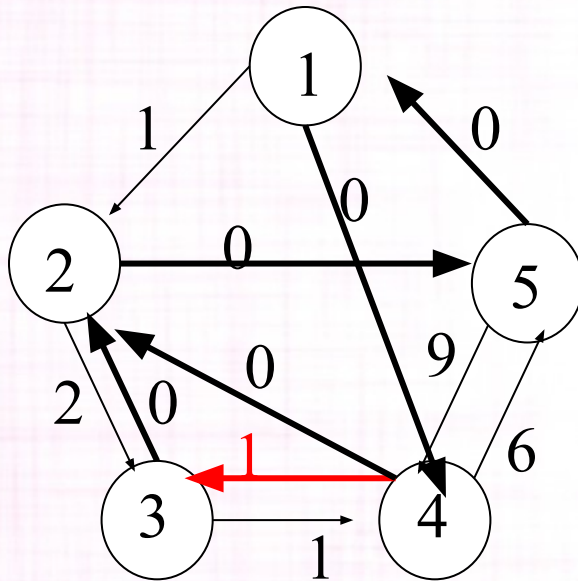
# Bound on TSP Tour



$$\begin{pmatrix} 999 & 1 & 999 & 0 & 999 \\ 999 & 999 & 2 & 999 & 0 \\ 999 & 0 & 999 & 1 & 999 \\ 999 & 0 & 1 & 999 & 6 \\ 0 & 999 & 999 & 9 & 999 \end{pmatrix}$$

Reduce all rows.

# Bound on TSP Tour



$$\begin{pmatrix} 999 & 1 & 999 & 0 & 999 \\ 999 & 999 & 1 & 999 & 0 \\ 999 & 0 & 999 & 1 & 999 \\ 999 & 0 & 0 & 999 & 6 \\ 0 & 999 & 999 & 9 & 999 \end{pmatrix}$$

Then reduce column #3. Now we have a tight bound.

# Using this bound for TSP in B&B

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 1   | 999 | 0   | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 0   | 999 | 1   | 999 |
| 999 | 0   | 0   | 999 | 6   |
| 0   | 999 | 999 | 9   | 999 |

bound = 21

1to2

1to3

1to4

1to5

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 1   | 999 | 0   | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 0   | 999 | 1   | 999 |
| 999 | 0   | 0   | 999 | 6   |
| 0   | 999 | 999 | 9   | 999 |

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 1   | 999 | 0   | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 0   | 999 | 1   | 999 |
| 999 | 0   | 0   | 999 | 6   |
| 0   | 999 | 999 | 9   | 999 |

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 1   | 999 | 0   | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 0   | 999 | 1   | 999 |
| 999 | 0   | 0   | 999 | 6   |
| 0   | 999 | 999 | 9   | 999 |

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 1   | 999 | 0   | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 0   | 999 | 1   | 999 |
| 999 | 0   | 0   | 999 | 6   |
| 0   | 999 | 999 | 9   | 999 |

bound = 21+1



# Using this bound for TSP in B&B

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 1   | 999 | 0   | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 0   | 999 | 1   | 999 |
| 999 | 0   | 0   | 999 | 6   |
| 0   | 999 | 999 | 9   | 999 |

bound = 21

1to2

1to3

1to4

1to5

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 1   | 999 | 0   | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 0   | 999 | 1   | 999 |
| 999 | 0   | 0   | 999 | 6   |
| 0   | 999 | 999 | 9   | 999 |

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 1   | 999 | 0   | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 0   | 999 | 1   | 999 |
| 999 | 0   | 0   | 999 | 6   |
| 0   | 999 | 999 | 9   | 999 |

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 1   | 999 | 0   | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 0   | 999 | 1   | 999 |
| 999 | 0   | 0   | 999 | 6   |
| 0   | 999 | 999 | 9   | 999 |

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 1   | 999 | 0   | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 0   | 999 | 1   | 999 |
| 999 | 0   | 0   | 999 | 6   |
| 0   | 999 | 999 | 9   | 999 |

bound = 21+1

# Focus: going from 1 to 2

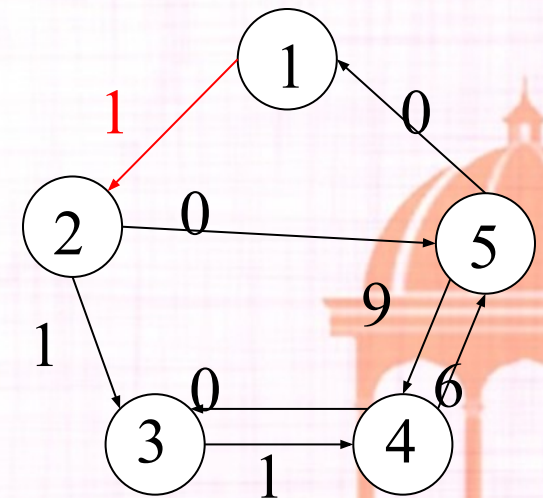
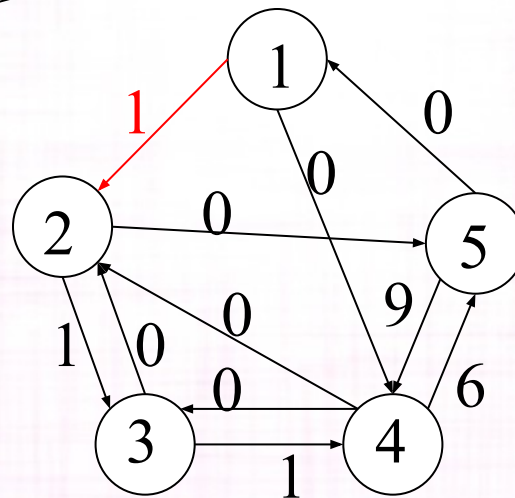
|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 1   | 999 | 0   | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 0   | 999 | 1   | 999 |
| 999 | 0   | 0   | 999 | 6   |
| 0   | 999 | 999 | 9   | 999 |

bound = 21

1to2

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 999 | 999 | 999 | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 999 | 999 | 1   | 999 |
| 999 | 999 | 0   | 999 | 6   |
| 0   | 999 | 999 | 9   | 999 |

bound = 21+1



Add extra cost from 1 to 2, exclude edges from 1 or into 2.

# Focus: going from 1 to 2

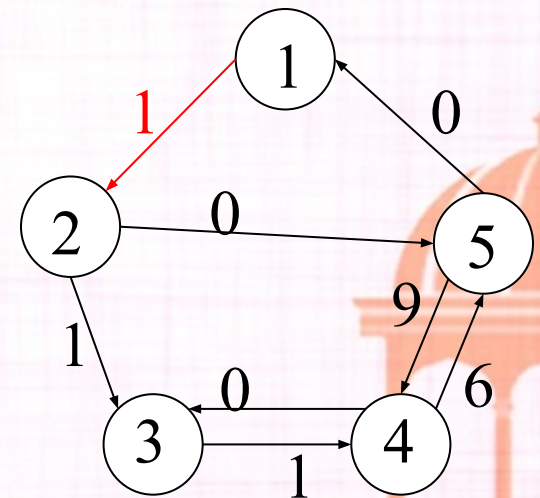
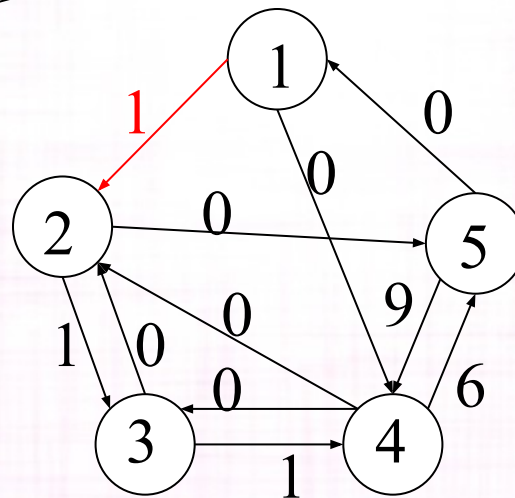
|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 1   | 999 | 0   | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 0   | 999 | 1   | 999 |
| 999 | 0   | 0   | 999 | 6   |
| 0   | 999 | 999 | 9   | 999 |

bound = 21

1to2

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 999 | 999 | 999 | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 999 | 999 | 1   | 999 |
| 999 | 999 | 0   | 999 | 6   |
| 0   | 999 | 999 | 9   | 999 |

bound = 21+1+1



No edges into vertex 4 w/ 0 reduced cost.



# Focus: going from 1 to 2

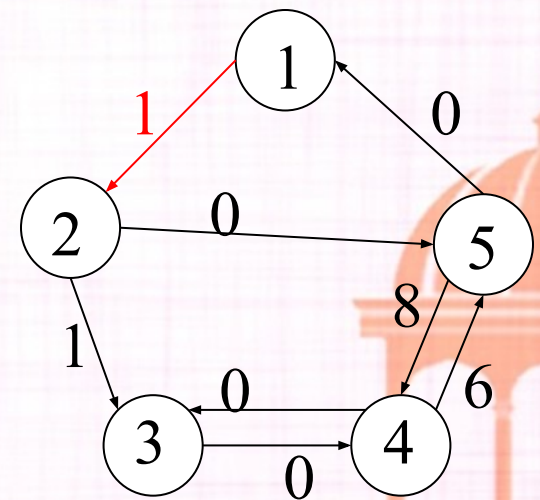
|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 1   | 999 | 0   | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 0   | 999 | 1   | 999 |
| 999 | 0   | 0   | 999 | 6   |
| 0   | 999 | 999 | 9   | 999 |

bound = 21

1to2

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 999 | 999 | 999 | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 999 | 999 | 0   | 999 |
| 999 | 999 | 0   | 999 | 6   |
| 0   | 999 | 999 | 8   | 999 |

bound = 21+1+1



Add cost of reducing edge into vertex 4.

# Bounds for other choices.

1to2(23), 1to4(21)

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 1   | 999 | 0   | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 0   | 999 | 1   | 999 |
| 999 | 0   | 0   | 999 | 6   |
| 0   | 999 | 999 | 9   | 999 |

bound = 21

1to2

1to3

1to4

1to5

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 999 | 999 | 999 | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 999 | 999 | 0   | 999 |
| 999 | 999 | 0   | 999 | 6   |
| 0   | 999 | 999 | 8   | 999 |

bound = 23

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 999 | 999 | 0   | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 0   | 999 | 1   | 999 |
| 999 | 0   | 0   | 999 | 6   |
| 0   | 999 | 999 | 9   | 999 |

bound = 999

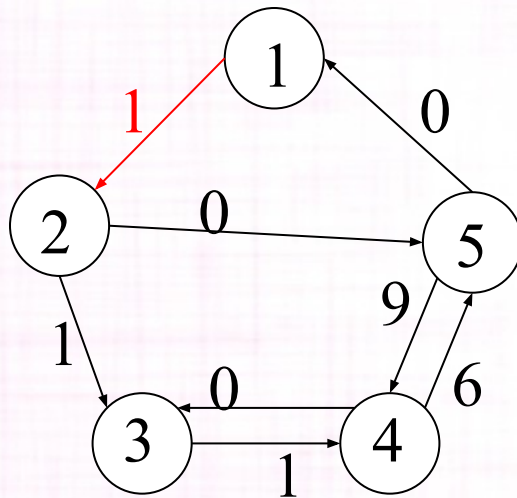
|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 999 | 999 | 999 | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 0   | 999 | 999 | 999 |
| 999 | 0   | 0   | 999 | 6   |
| 0   | 999 | 999 | 999 | 999 |

bound = 21

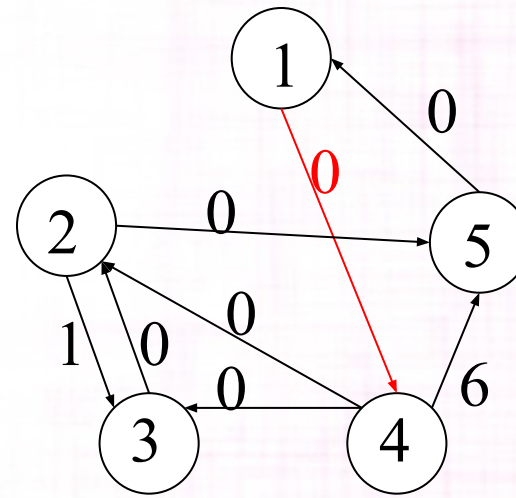
|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 1   | 999 | 0   | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 0   | 999 | 1   | 999 |
| 999 | 0   | 0   | 999 | 6   |
| 0   | 999 | 999 | 9   | 999 |

bound = 999

# Leaves us with Two Possibilities on Priority Queue



bound = 23



bound = 21

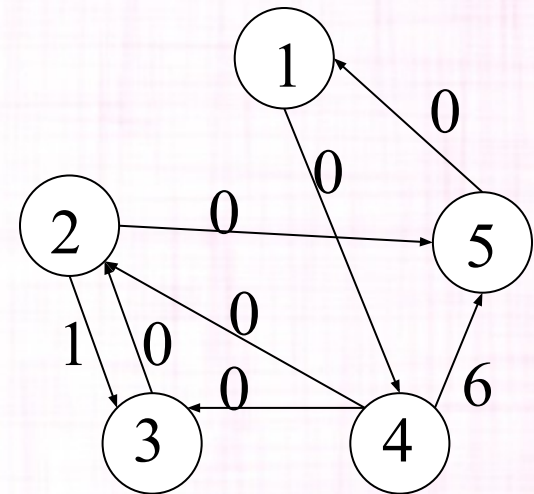


# Leaving Vertex 4

4to2(22), 4to3(21)  
4to5(28), 1to2(23),

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 999 | 999 | 999 | 999 |
| 999 | 999 | 1   | 999 | 0   |
| 999 | 0   | 999 | 999 | 999 |
| 999 | 0   | 0   | 999 | 6   |
| 0   | 999 | 999 | 999 | 999 |

bound = 21



4to2

4to3

4to5

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 999 | 999 | 999 | 999 |
| 999 | 999 | 0   | 999 | 0   |
| 999 | 999 | 999 | 999 | 999 |
| 999 | 999 | 999 | 999 | 999 |
| 0   | 999 | 999 | 999 | 999 |

bound = 22

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 999 | 999 | 999 | 999 |
| 999 | 999 | 999 | 999 | 0   |
| 999 | 0   | 999 | 999 | 999 |
| 999 | 999 | 999 | 999 | 999 |
| 0   | 999 | 999 | 999 | 999 |

bound = 21

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 999 | 999 | 999 | 999 |
| 999 | 999 | 0   | 999 | 999 |
| 999 | 0   | 999 | 999 | 999 |
| 999 | 999 | 999 | 999 | 999 |
| 0   | 999 | 999 | 999 | 999 |

bound = 28

# Leaving Vertex 3

4to2(22), 3to2(21)  
1to2(23),

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 999 | 999 | 999 | 999 |
| 999 | 999 | 999 | 999 | 0   |
| 999 | 0   | 999 | 999 | 999 |
| 999 | 999 | 999 | 999 | 999 |
| 0   | 999 | 999 | 999 | 999 |

bound = 21

3to2

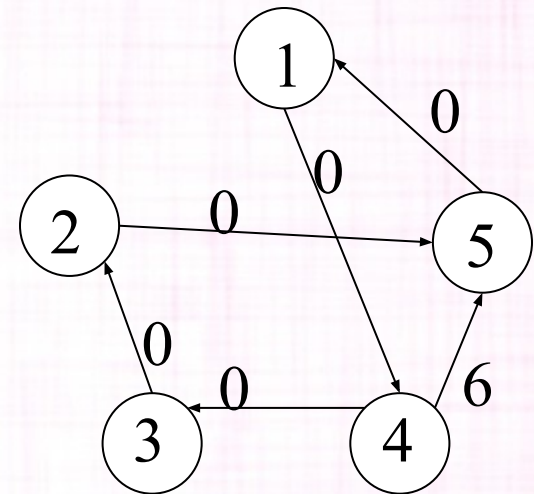
3to5

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 999 | 999 | 999 | 999 |
| 999 | 999 | 999 | 999 | 0   |
| 999 | 999 | 999 | 999 | 999 |
| 999 | 999 | 999 | 999 | 999 |
| 0   | 999 | 999 | 999 | 999 |

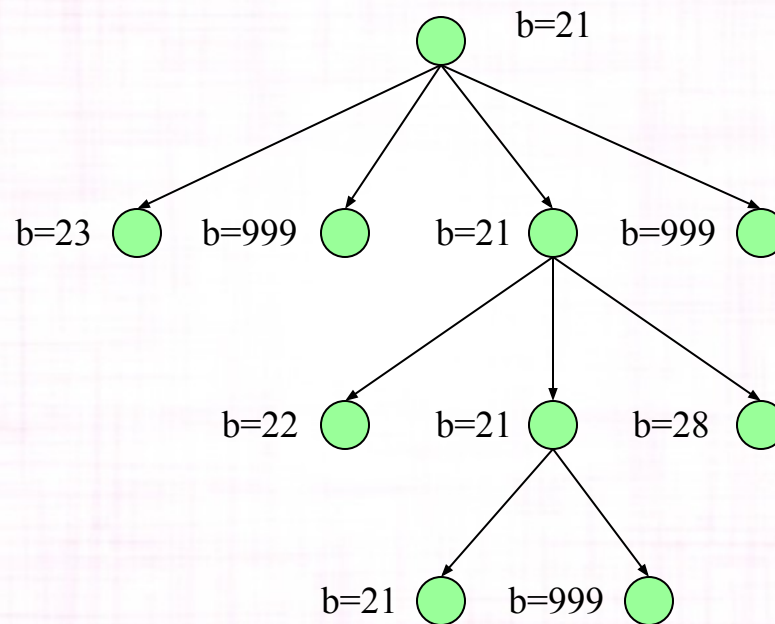
bound = 21

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 999 | 999 | 999 | 999 | 999 |
| 999 | 999 | 999 | 999 | 999 |
| 999 | 999 | 999 | 999 | 999 |
| 999 | 999 | 999 | 999 | 999 |
| 0   | 999 | 999 | 999 | 999 |

bound = 999



# Search Tree for This Problem







## THE 0/1 KNAPSACK PROBLEM

- Positive integer  $P_1, P_2, \dots, P_n$  (profit)  
 $W_1, W_2, \dots, W_n$  (weight)  
 $M$  (capacity)

$$\text{maximize } \sum_{i=1}^n P_i X_i$$

$$\text{subject to } \sum_{i=1}^n W_i X_i \leq M \quad X_i = 0 \text{ or } 1, i = 1, \dots, n.$$

The problem is modified:

$$\text{minimize } - \sum_{i=1}^n P_i X_i$$



# 0/1 Knapsack using Least Cost Branch and Bound



Given  $N$  items with weights  $W$  and Profit  $P$  and a knapsack with capacity  $M$ , select the items such that:

1. The sum of weights taken into the knapsack is less than or equal to  $C$ .
2. The sum of **profit** of the items in the knapsack is **maximum** among all the possible combinations.





**MIT-ADT**  
**UNIVERSITY**  
PUNE, INDIA







**MIT-ADT**  
**UNIVERSITY**  
PUNE, INDIA





**MIT-ADT**  
**UNIVERSITY**  
PUNE, INDIA





**MIT-ADT**  
**UNIVERSITY**  
PUNE, INDIA







## THE 0/1 KNAPSACK PROBLEM [CONT..]

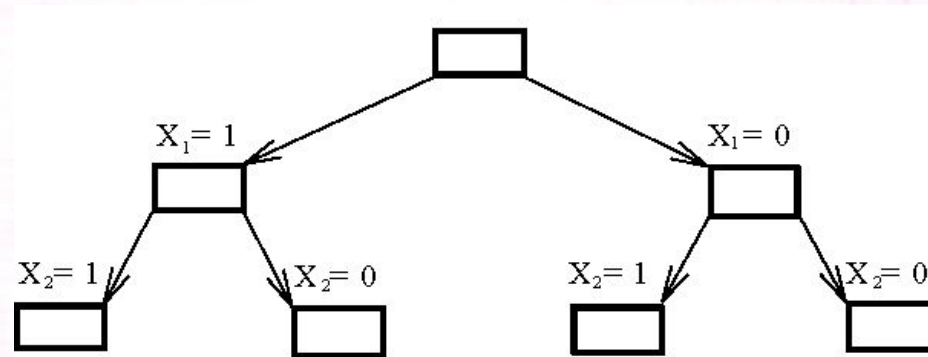


Fig. 6-27 The Branching Mechanism in the Branch-and-Bound Strategy to Solve 0/1 Knapsack Problem.



## HOW TO FIND THE UPPER BOUND?

- Ans: by quickly finding a feasible solution in a **greedy manner**: starting from the smallest available  $i$ , scanning towards the largest  $i$ 's until  $M$  is exceeded. The upper bound can be calculated.





## THE 0/1 KNAPSACK PROBLEM

- E.g.  $n = 6, M = 34$

|       |    |    |   |    |    |   |
|-------|----|----|---|----|----|---|
| $i$   | 1  | 2  | 3 | 4  | 5  | 6 |
| $P_i$ | 6  | 10 | 4 | 5  | 6  | 4 |
| $W_i$ | 10 | 19 | 8 | 10 | 12 | 8 |

$$(P_i/W_i \geq P_{i+1}/W_{i+1})$$

- A feasible solution:  $X_1 = 1, X_2 = 1, X_3 = 0, X_4 = 0,$   
 $X_5 = 0, X_6 = 0$   
 $-(P_1 + P_2) = -16$  (upper bound)  
Any solution higher than -16 can not be an optimal solution.





## HOW TO FIND THE LOWER BOUND?

- Ans: by relaxing our restriction from  $X_i = 0$  or  $1$  to  $0 \leq X_i \leq 1$  (knapsack problem)

Let  $-\sum_{i=1}^n P_i X_i$  be an optimal solution for 0/1

knapsack problem and  $-\sum_{i=1}^n P_i X'_i$  be an optimal

solution for **fractional knapsack problem**. Let

$$Y = -\sum_{i=1}^n P_i X_i, \quad Y' = -\sum_{i=1}^n P_i X'_i.$$

$$\Rightarrow Y' \leq Y$$





Dept. of Computer Science &  
Engineering.



**MIT-ADT**  
**UNIVERSITY**  
PUNE, INDIA

**THANK YOU**

